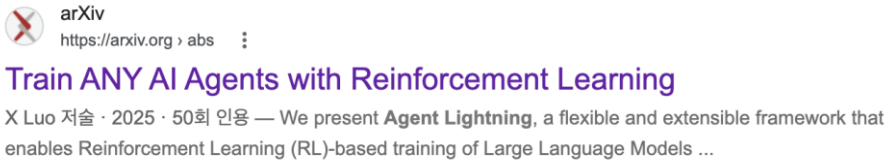
Agent Lightning: Train ANY AI Agents with Reinforcement Learning

Xufang Luo∗, Yuge Zhang∗, Zhiyuan He∗, Zilong Wang, Siyun Zhao, Dongsheng Li,
Luna K. Qiu, Yuqing Yang, Microsoft Research

2026. 08. 13
Learning Agents 강화학습 논문 리뷰 스터디
Minkyoung Kim

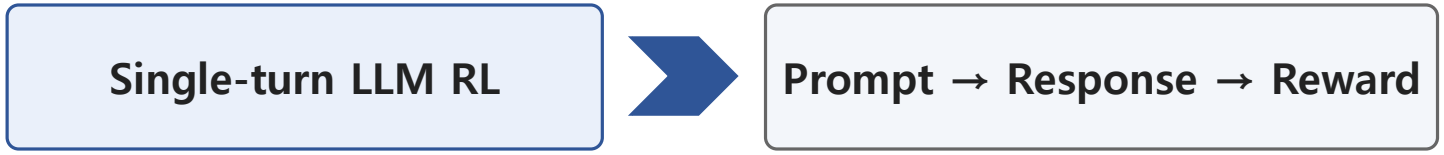


50회 인용(`26.08.09)

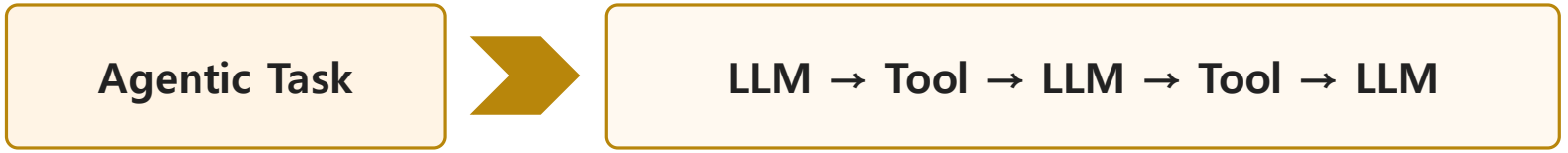
Introduction

- Extending the RL paradigm to agents introduces substantial challenges in both algorithm design and system implementation.

- **Traditional single-turn LLM RL**
  - For static, single-call tasks: previous (preference alignment, mathematical reasoning)



- **Limitation**
  - Complexity: execution involves multiple LLM invocations, external tools
  - Diversity: various applications



Agent Lightning

- Flexible and extensible framework enables RL-based training of LLMs for **ANY Agents**
- Complete **decoupling between agent execution and RL training**, almost ZERO code m odifications.

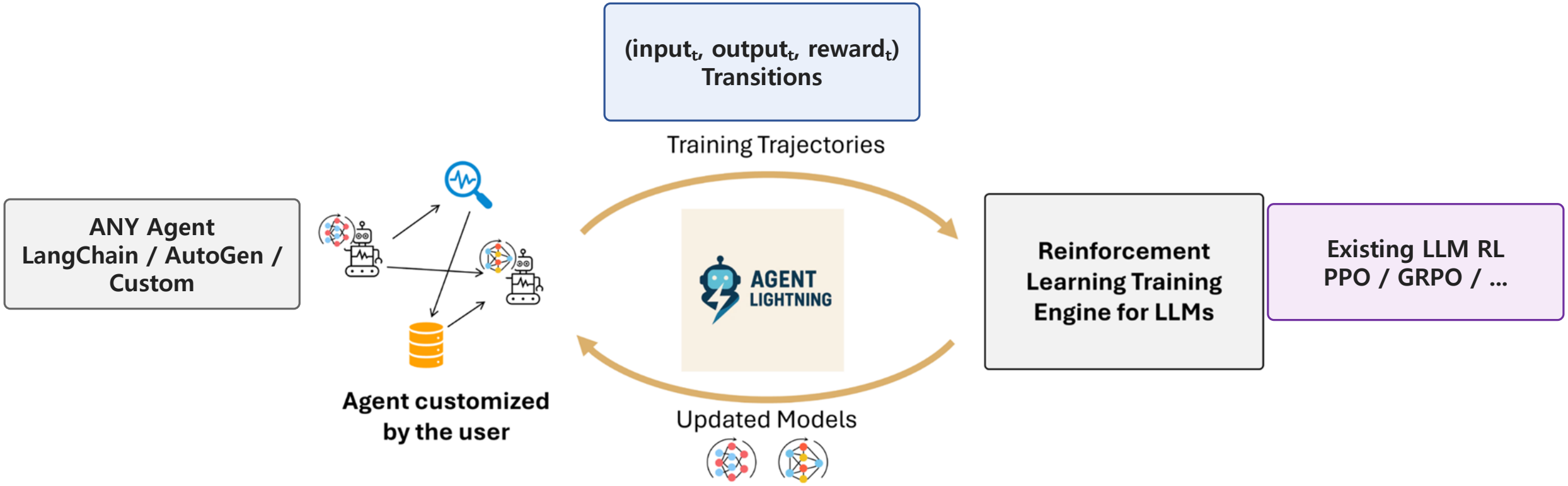


Figure 1: Overview of *Agent Lightning*, a flexible and extensible framework that enables reinforcement learning of LLMs for ANY AI agents.

Modern AI Agents

- Defining AI Agent is challenging due to their diversity and rapid evolution.
- ***An AI agents*** *is a software system that incorporates one or more LLM calls within its execution.*

→ **Provide abstract formulation** that serves as a common understanding of AI agents

Modern AI Agents

- Components for agents: **LLM, Tools**
- **LLM**
  - serves as the core reasoning and generation engines within agents
  - set of LLM used by an agent as $\mathcal{M} = \{M_i\}_{i=1}^{m}$
    - *m*: the number of LLMs in the agents
  - can link the model's name to its parameter θ_i M_i
- **Tools**
  - functionalities that the agent invokes to perform *specific tasks*
  - e.g. database, executing code, interacting with APIs
  - *t*: the number of tools $\mathcal{T} = \{T_j\}_{j=1}^{t}$

Modern AI Agents

- Orchestration
  - Using **component set as functional building blocks**, $\mathcal{M} \cup \mathcal{T}$ orchestration is customized by the user based on task requirements.
    - (e.g. execution flow, dependencies, order among components)
  - is often neither fixed nor deterministic.

Unified Data Interface

- For RL training, how this data feeds into RL training algorithms for ANY AI agents

No need to reconstruct the complete execution DAG(Directed acyclic graph)
→ Using the concept in MDP!

- Transition

State = execution snapshot (semantic variables)

Call = (meta, input, output)

Reward = per-call or terminal feedback

- Effect
  - Abstracts away the underlying orchestration logic and agent framework detail → ANY agent!

Unified Data Interface

- State and Call
  - State: Snapshot of the agent execution
  - for the *k*-th execution of task *x*,
    state at timestep *t* containing *V* variables is represented as:

$$\texttt{state}_t(x,k) = \left\{\texttt{variable}_i^{x,k,t}\right\}_{i=1}^{V}.$$

  - State changes continuously during agent execution.
    - ***Semantic Variable****: a key variable used/modified by components that captures execution semantics*

  - *k*-th execution of task *x* consists of *N* component invocations/calls:

$$\texttt{execution}(x,k) = \left\{\texttt{call}_i^{x,k}\right\}_{i=1}^{N}, \qquad C_i \in \mathcal{M} \cup \mathcal{T}$$

$$\texttt{call}_i^{x,k} = (\texttt{meta}_i^{x,k}, \texttt{input}_i^{x,k}, \texttt{output}_i^{x,k}), \quad \text{with } \texttt{output}_i^{x,k} = C_i(\texttt{input}_i^{x,k}).$$

Unified Data Interface

- State and Call
  - Meta: relevant meta information about the invocation.
    - C_i's name , type, version, API endpoint

$$C_i \in \mathcal{M} \cup \mathcal{T}$$

$$\texttt{call}_i^{x,k} = (\texttt{meta}_i^{x,k}, \texttt{input}_i^{x,k}, \texttt{output}_i^{x,k}), \quad \text{with } \texttt{output}_i^{x,k} = C_i(\texttt{input}_i^{x,k}).$$

- **Not all information in the state is necessary or visible to the component being called.**

Unified Data Interface

- Reward and Dataset
  - Each agent execution is augmented with scalar rewards $\{r_1, \ldots, r_N\}$, where each corresponds to the quality of *i*-th invocation. $r_i \in \mathbb{R}$

$$\texttt{execution}^R(x,k) = \left\{ \left(\texttt{call}_i^{x,k}, r_i^{x,k}\right) \right\}_{i=1}^{N}.$$

  - In many real-world scenarios,
    only a terminal reward r_N is available to assess the outcome of the entire execution.

Unified Data Interface - Example

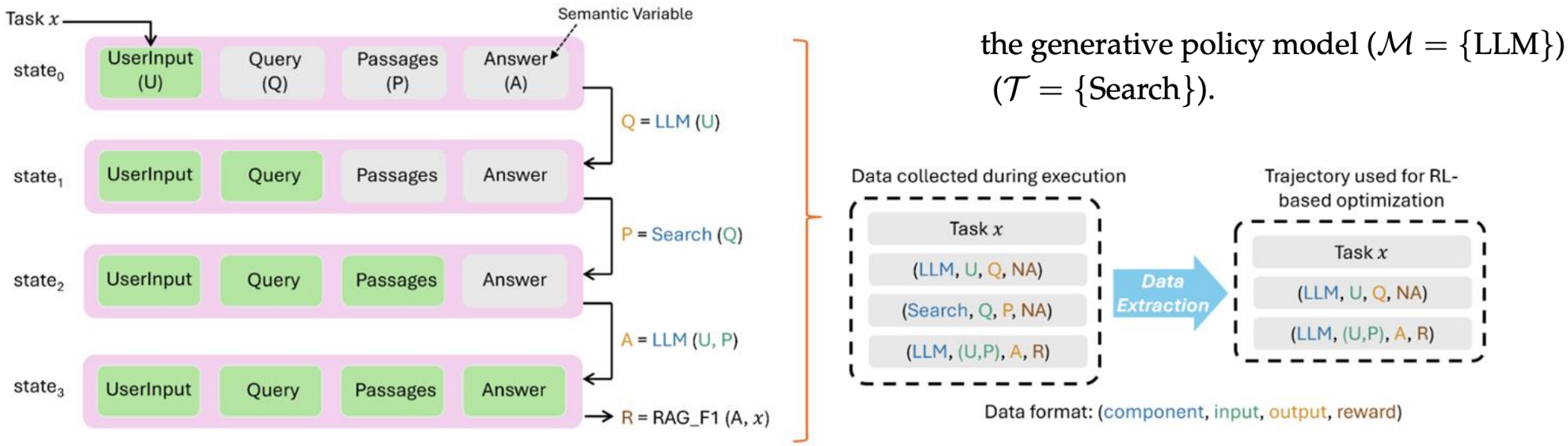


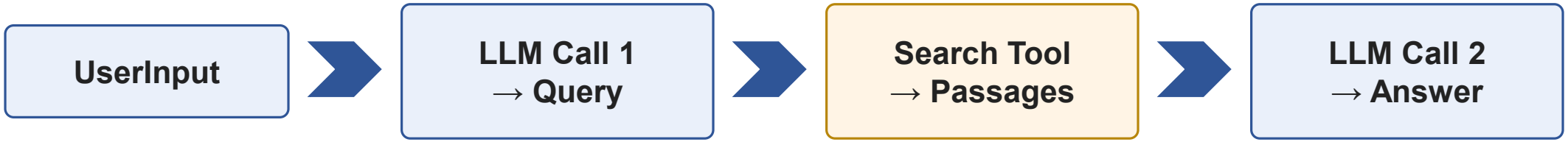
Figure 2: Illustration of the unified data interface in *Agent Lightning*. The left panel depicts the agent execution flow, where each state transition is represented by the update of semantic variables (green rectangles denote variables with valid values; gray rectangles indicate variables not yet assigned in the current state). The right panel presents the corresponding trajectory collected throughout the agent's execution, demonstrating how the unified data interface systematically captures all relevant transitions for RL-based optimization.

State = execution snapshot (semantic variables)

Call = (meta, input, output)

Reward = per-call or terminal feedback

Unified Data Interface - Example



State All semantic variables currently held by the agent

Observation The context actually provided to the current policy-LLM call

→ Hence POMDP: the LLM observes only the relevant part of the full state.

Markov Decision Process in Agents

- Partially observable markov decision process(POMDP) tuple *M = (S,O,A,P,R)*

- $\mathcal{S}$ is the space of states, corresponding to all possible states, i.e., $\texttt{state}_t \in \mathcal{S}$;
- $\mathcal{O}$ is the observation space, corresponding to all possible inputs to the policy LLM, i.e., $\texttt{input}_t \in \mathcal{O}$;
- $\mathcal{A}$ is the action space, where a single action is defined as the entire token sequence generated from a single invocation of the policy LLM, i.e., $\texttt{output}_t \in \mathcal{A}$.
- $\mathcal{T}(s'|s,a)$ defines the (usually unknown) transition dynamics to the new state;
- $\mathcal{R}(s,a)$ is the reward function, which maps state-action pairs to scalar rewards;

$$\underbrace{(y_{t,1}, y_{t,2}, \ldots, y_{t,N_t})}_{\text{one LLM response}} = \text{one agent-level action } a_t$$

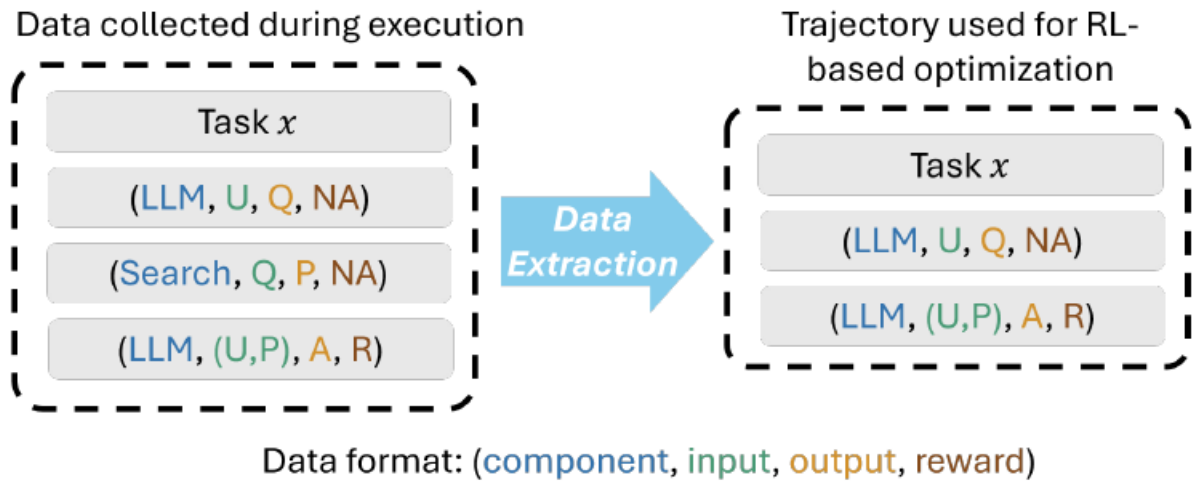
Data Extraction for RL

- Based on the MDP formulation, need to collect trajectories containing all policy LLM decisions and their associated rewards for RL.
  - π_θ is the LLM to be optimized.
  - T : the number of invocations in this trajectory

$$\text{execution}^{RL}(x,k) = \left\{ (\text{input}_t^{x,k}, \text{output}_t^{x,k}, r_t^{x,k}) \right\}_{t=1}^{T},$$

$$\text{with output}_t^{x,k} = \pi_\theta(\text{input}_t^{x,k}).$$

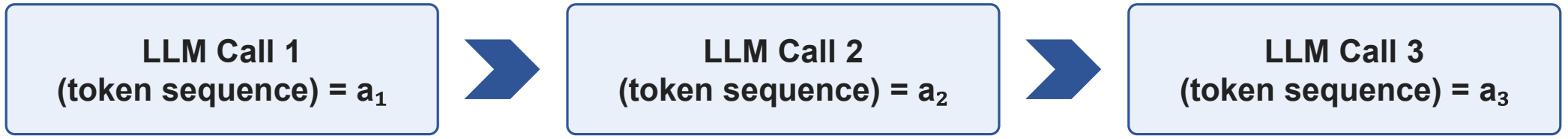
focusing solely on the current input and output of the LLM



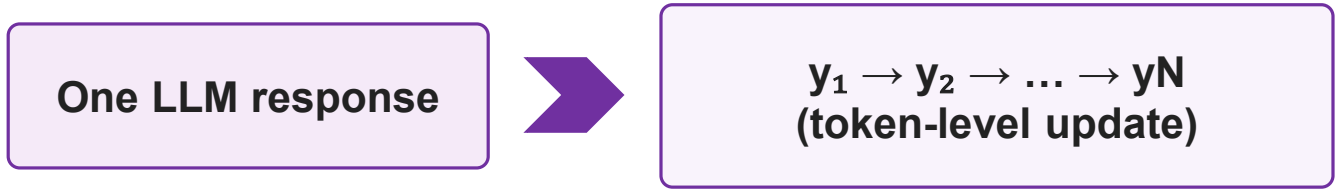
A Hierarchical RL for Optimizing LLMs in Agents

- **How do we connect an episode reward over multiple calls to learning at the response/token level?**

Agent-level MDP



Existing Single-Turn LLM RL

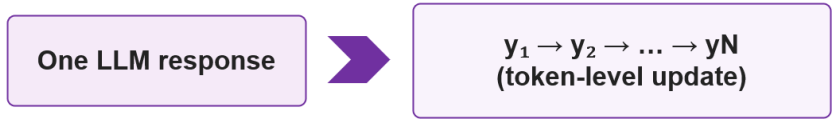


A Hierarchical RL for Optimizing LLMs in Agents

- Preliminary on Single-turn Reinforcement learning for LLMs
  - The model generates a response to a prompt in one pass.
  - Given a task x, an LLM generates a response output = $(y_1, y_2, \ldots, y_N)$ token by token from a policy π_θ, each toekn is treated as an action.
  - After the entire response is generated, a scalar reward r(x, output, $\hat{y}$) is assigned based on the correctness or quality of the solution, where $\hat{y}$ is the ground truth to task x.
  - The typical token-level loss can be simplified:
    - A_j : a token level advantage estimate
    - X: the task set

$$\mathcal{L}(\theta) = -\mathbb{E}_{x\sim\mathcal{X},\,\text{output}\sim\pi_\theta(\cdot|x)}\left[\sum_{j=1}^{N}\log\pi_\theta(y_j|x,y_{<j})\cdot A_j\right]$$

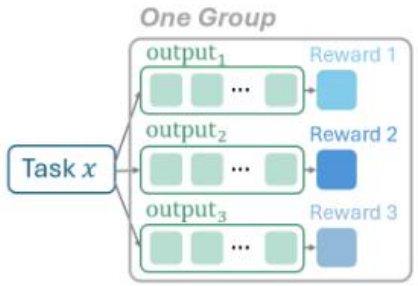
Existing Single-Turn LLM RL



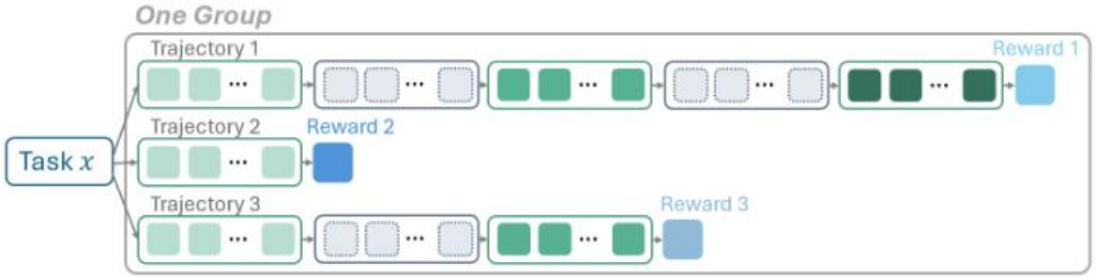
A Hierarchical RL for Optimizing LLMs in Agents

- Preliminary on Single-turn Reinforcement learning for LLMs

$$\mathcal{L}(\theta) = -\mathbb{E}_{x\sim\mathcal{X},\,\text{output}\sim\pi_\theta(\cdot|x)}\left[\sum_{j=1}^{N}\log\pi_\theta(y_j|x,y_{<j})\cdot A_j\right]$$



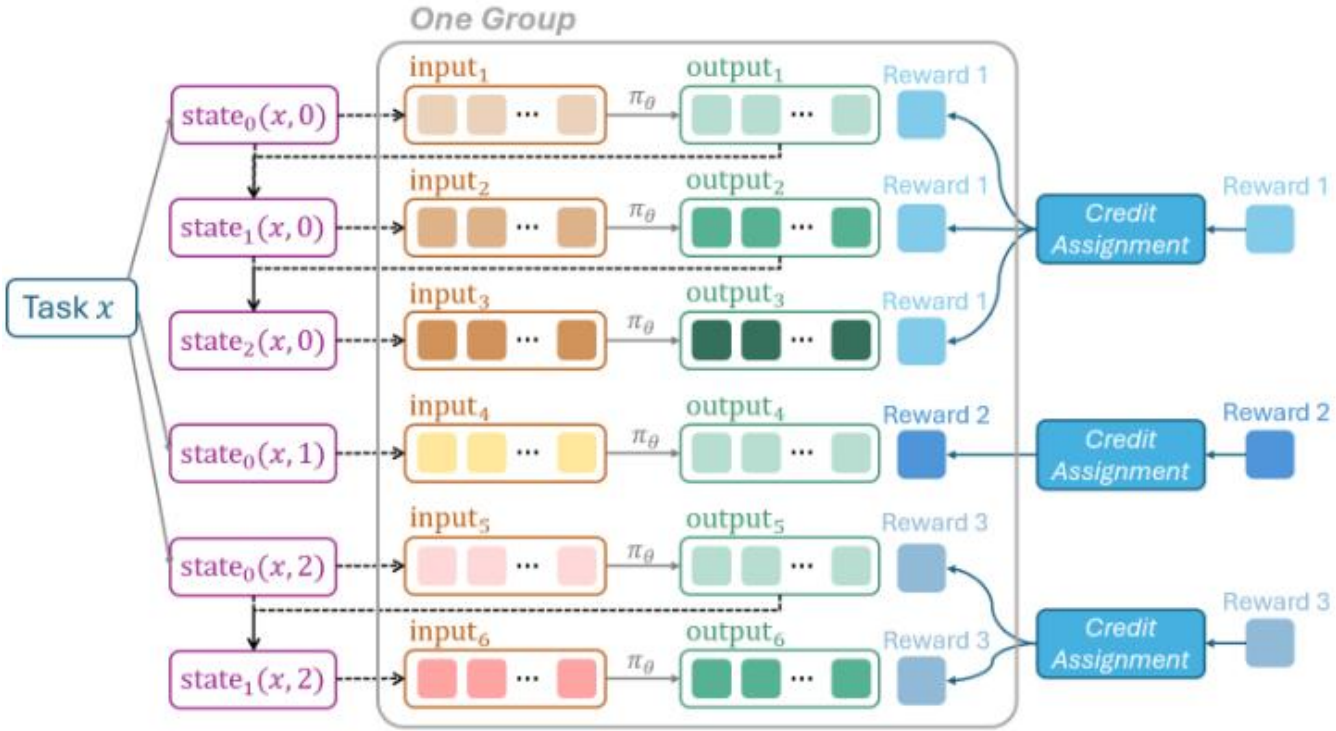
(a) Single-call GRPO



(b) Previous multi-turn GRPO

A Hierarchical RL for Optimizing LLMs in Agents

- Extend to Agent Scenario via Lightning RL
  - Most existing RL algorithms for LLMs are designed for single-turn interactions,
  - focusing on optimizing token generation within a single response.



(c) LightningRL (ours)

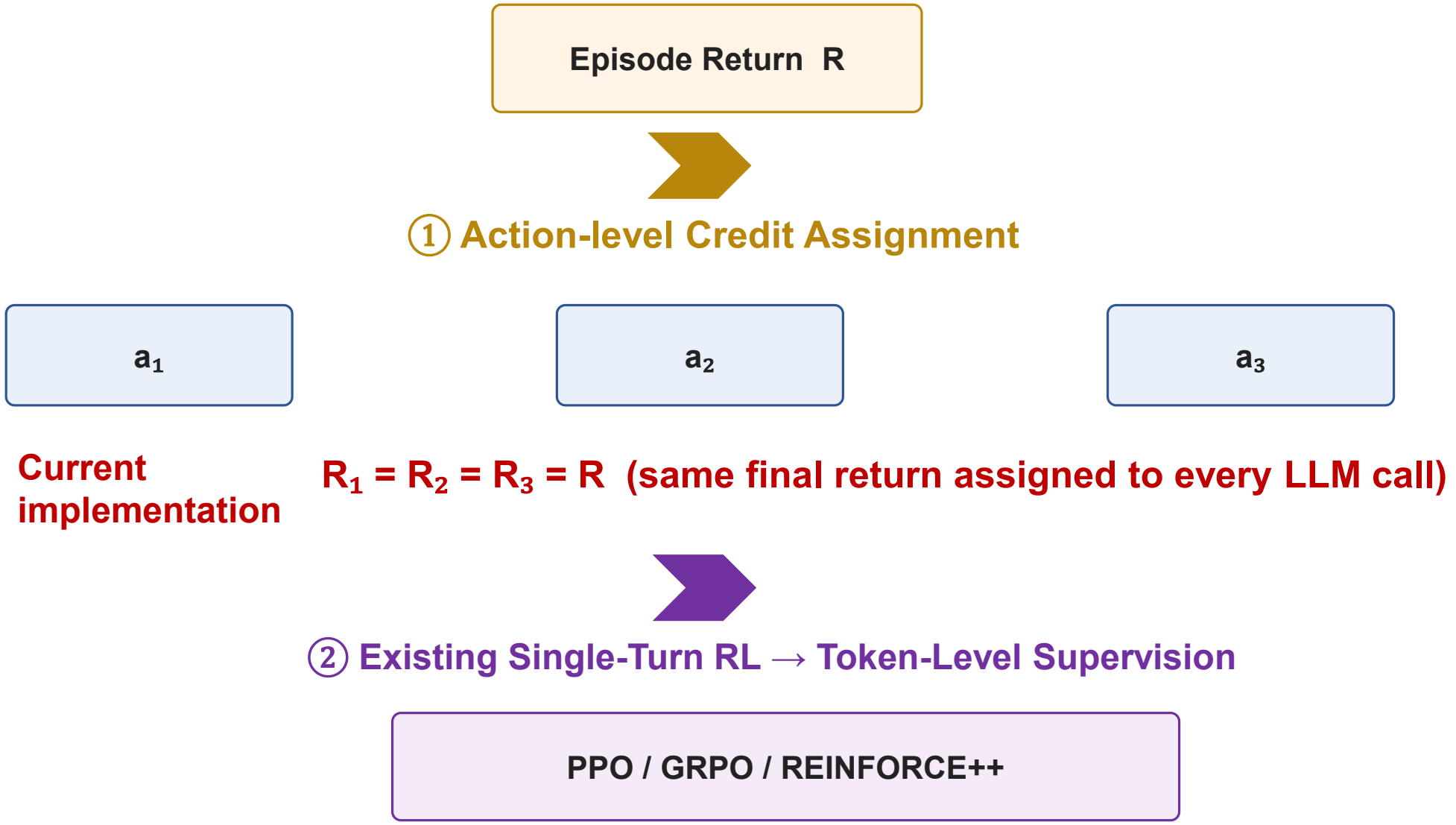
$$\texttt{execution}^{RL}(x,k) = \left\{(\texttt{input}_t^{x,k}, \texttt{output}_t^{x,k}, r_t^{x,k})\right\}_{t=1}^{T},$$

$$\text{with } \texttt{output}_t^{x,k} = \pi_\theta(\texttt{input}_t^{x,k}).$$

Episode-level return R is first assigned across actions by a credit assignment module

LightningRL simply assumes each action within the episode has the same value, equal to the final return R.

A Hierarchical RL for Optimizing LLMs in Agents



A Hierarchical RL for Optimizing LLMs in Agents

- Advantages of Hierarchical RL
  - Allows direct use of any single-turn RL algorithm without modification, especially value-free methods that are typically lightweight and efficient.
  - Enables flexible construction of policy observations.
    - previous methods often concatenated all turns in a trajectory into a single response.
  - Compared to the masking strategy, Lightning RL offers a more robust and scalable implementation.
    - agent trajectories → transition → aligning with the LLM's input

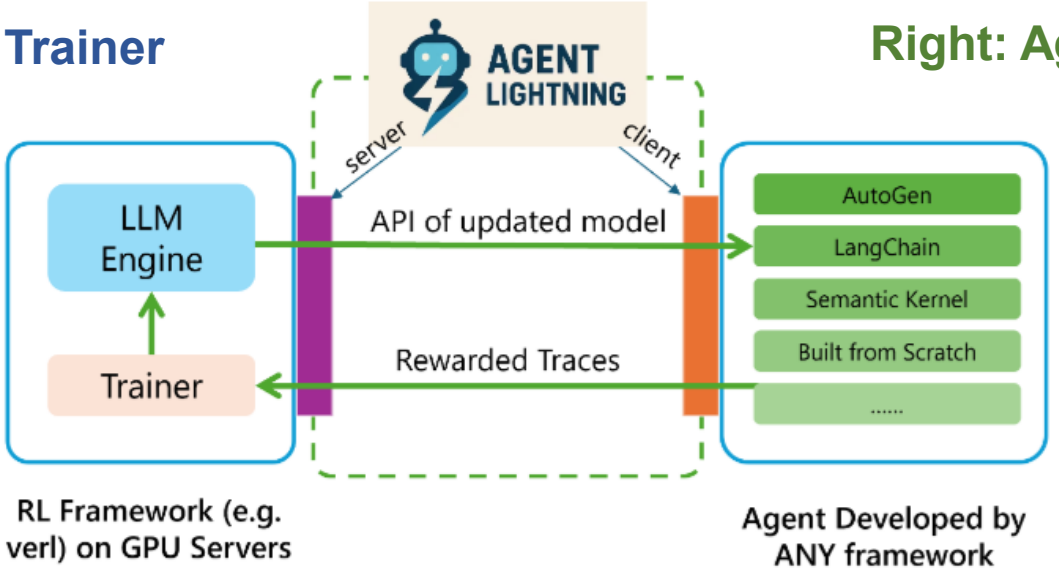
- Current limitation
  - No sophisticated action-specific credit assignment.
  - Terminal reward is broadcast identically to all actions.

Training-Agent Disaggregation(TA Disaggregation)

- Lightning Server
  - Functions as the controller of the RL training system.
  - Managing the training process and exposing an OpenAI-like API of the updated model to the client.
- Lightning Client
  - Communication with server for data transmission and reception
  - Runs the agent and performs data collection, serving as the agent runtime.

Left: Trainable Policy LLM + Trainer

Right: Agent Orchestration + Tools



Updated-model API → Agent Execution → Rewarded Traces → Trainer → Weight Update

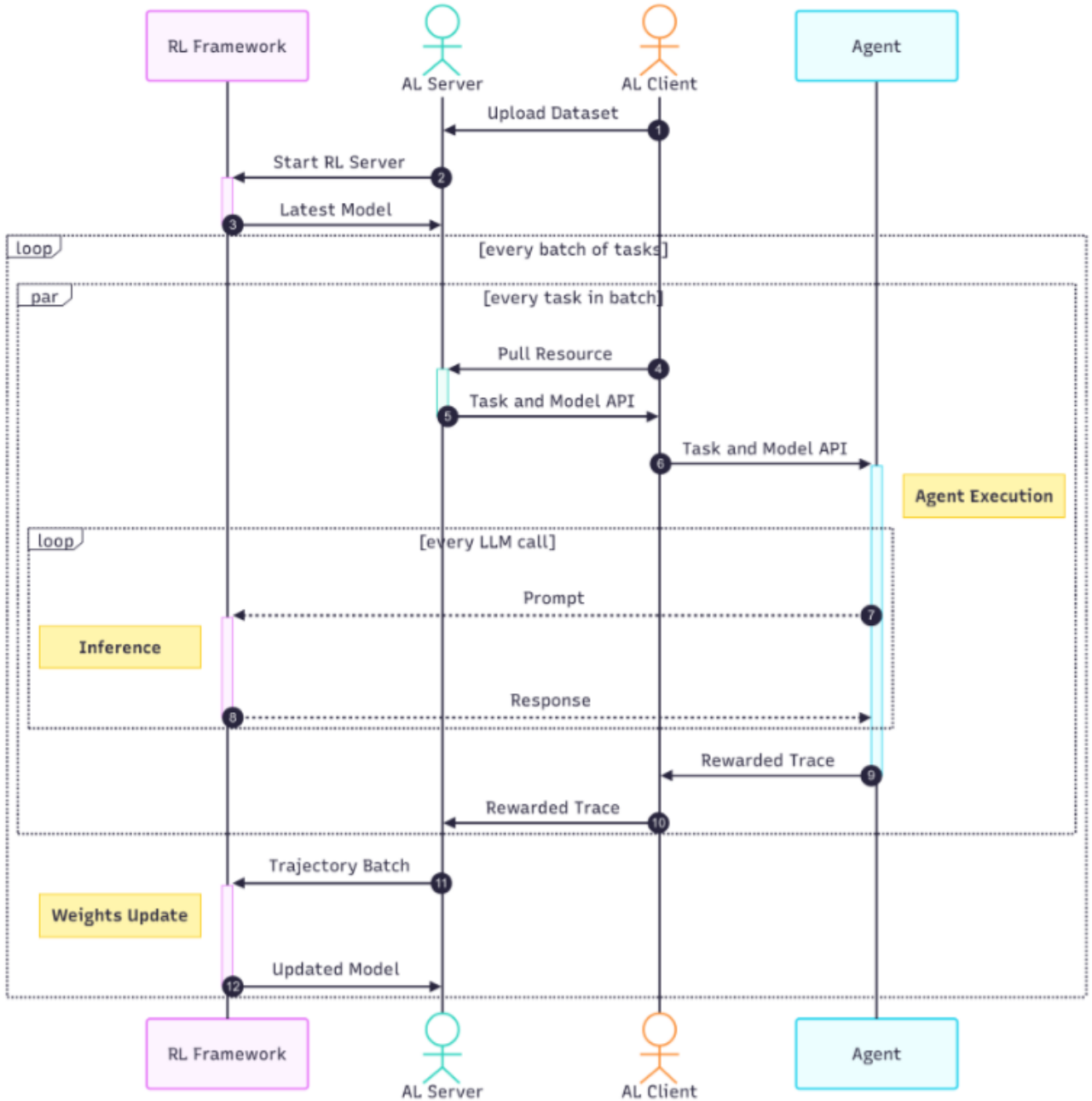
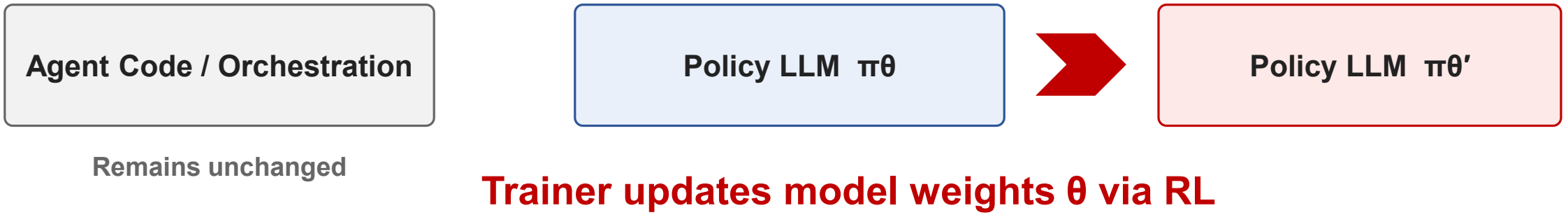


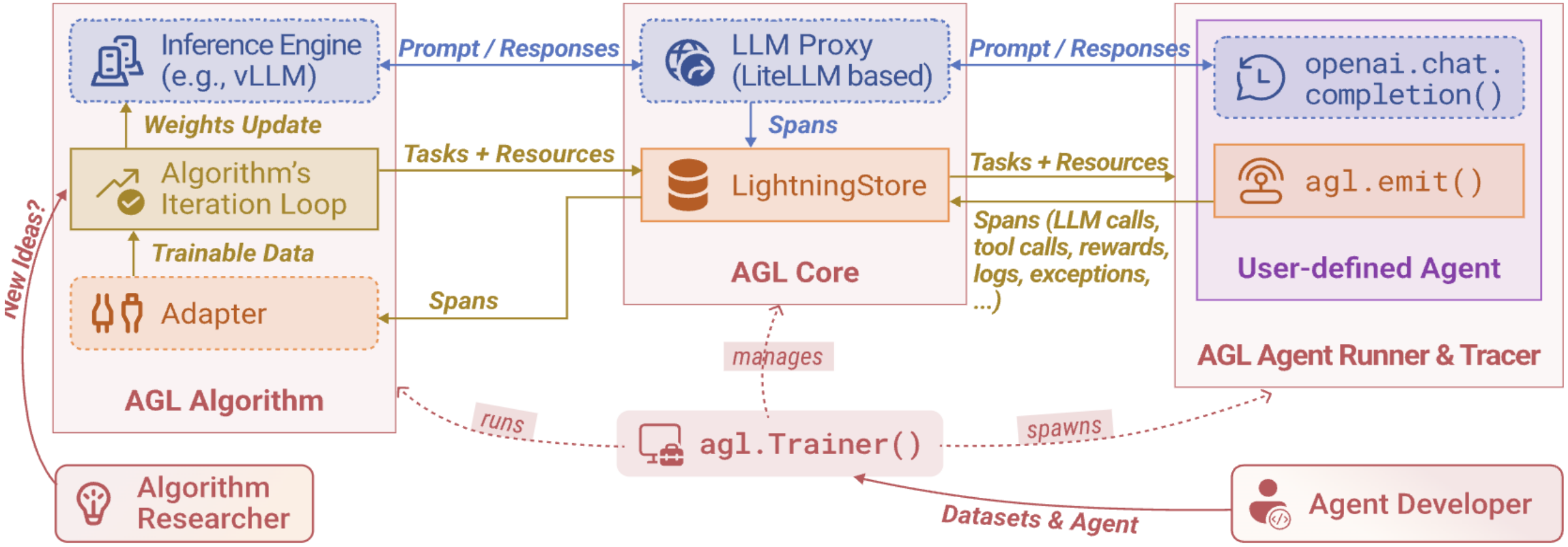
Figure 8: Process diagram

Training-Agent Disaggregation(TA Disaggregation)

- This does not directly update the internal weights of a commercial LLM API.
- The primary target is a trainable/open-weight policy model.
- The agent accesses the model under training through an OpenAI-like endpoint.



Implementation Detail: AGL Core / Trace / Adapter



Agent → LLM Proxy → Inference Engine

Spans → LightningStore → Adapter → Trainable Data

RL Loop → Weight Update

Experiments

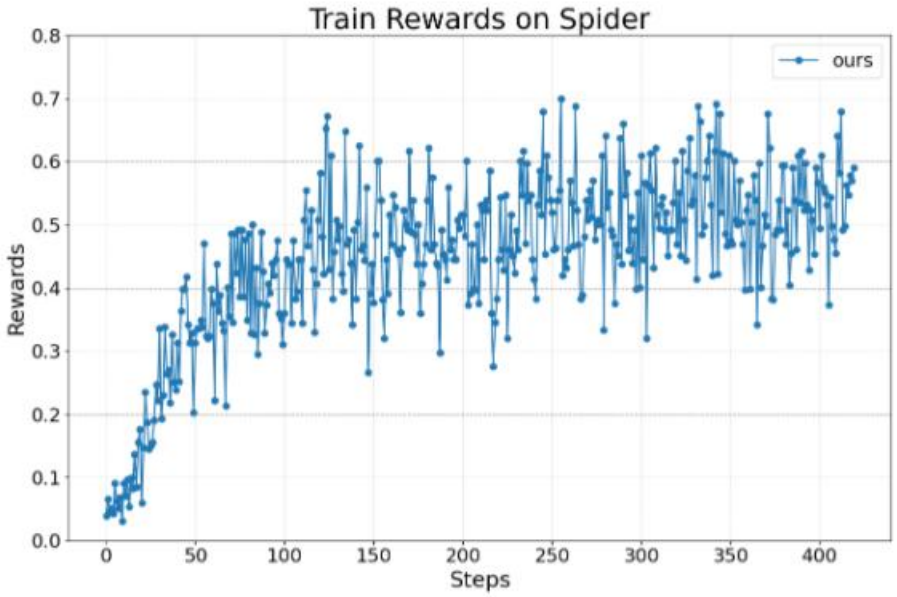
- Text-to-SQL: Multi-Agent system에서 원하는 Agent의 transition만 선택해서 학습
- Open domain QA: Dynamic RAG Agent 학습
- Math QA: Tool-use Agent 학습
- Base model: Lamma-3.2-3B-Instrunction(Meta, 2024)

Table 1: Summary of the tasks and settings in experiments.

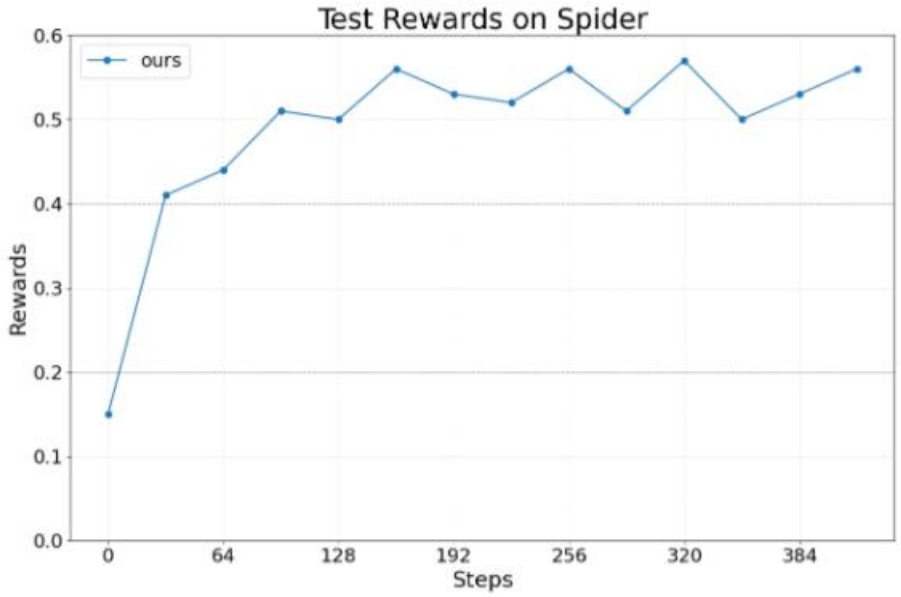
Task	**Text-to-SQL**	**Open domain QA**	**Math QA**
Framework	LangChain	OpenAI Agents SDK	AutoGen
Dataset	Spider	MuSiQue	Calc-X
Tool used	SQL executor	Wikipedia retriever	Calculator
Num. of agents	3	1	1
Num. of tuned agents	2	1	1

Experiments

- Text-to-SQL
  - **SQL writer agent**: generate a SQL query, which is then executed
  - Checking Agent: correctness of the SQL query and sufficiency of the retrieved information
  - **Re-writing Agent**: rewrite query or generate an answer



(a) Train reward

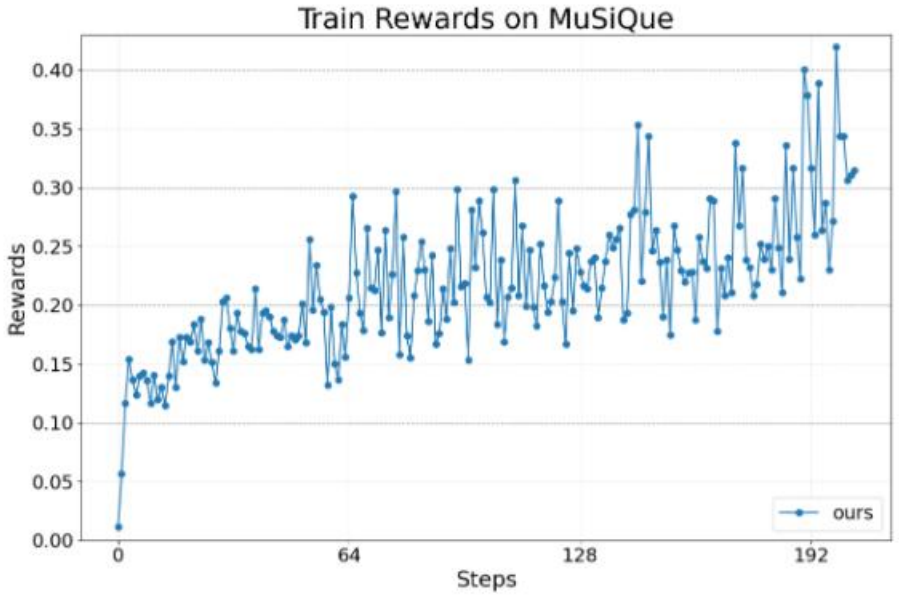


(b) Test reward

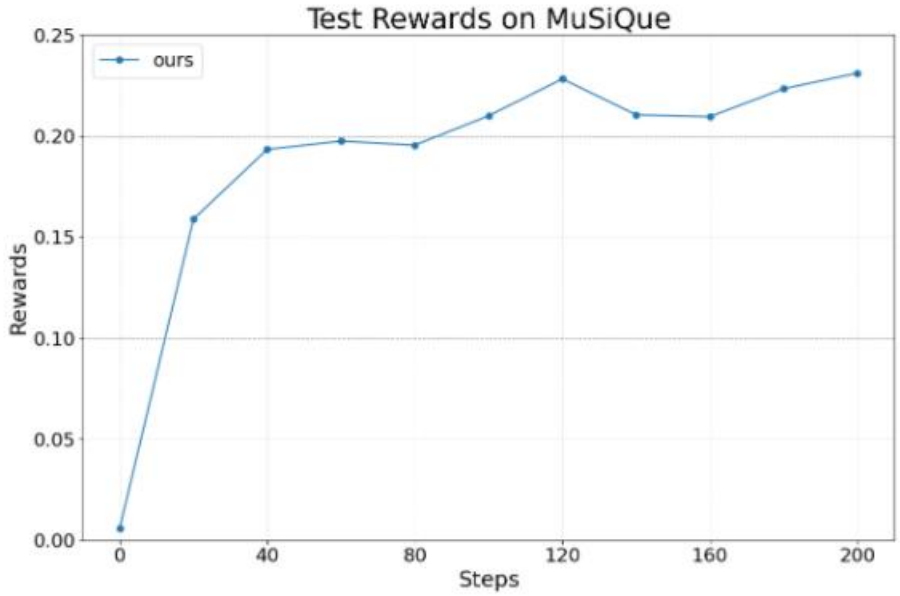
Can Agent Lightning selectively optimize LLM agents inside a multi-agent workflow?

Experiments

- Open domain QA: Dynamic RAG Agent 학습
  - Agent는 문제에 따라 retrieval과 LLM interaction 실행 흐름이 달라짐(**Dynamic orchestration**)
  - Reward =0.9×Correctness F1+0.1×Format



(a) Train reward

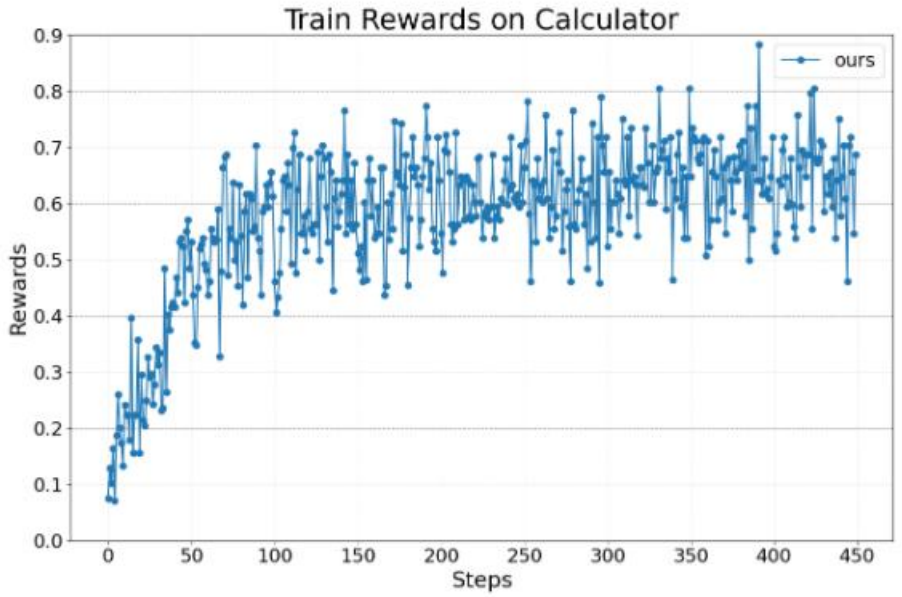


(b) Test reward

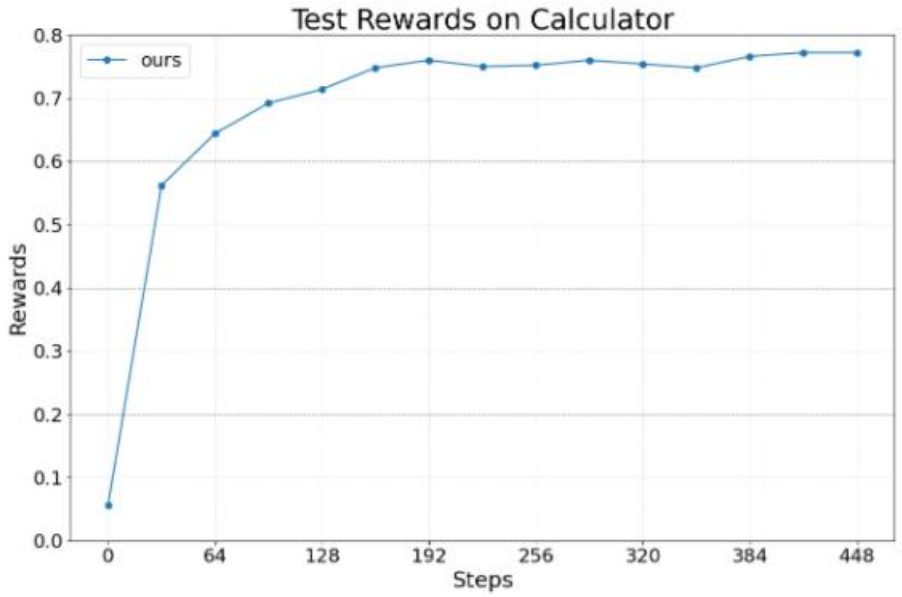
Can RL train an agent whose retrieval / LLM execution path changes dynamically by task?

Experiments

- Math QA: Tool-use Agent 학습
  - 외부 Tool을 사용하는 Agent의 행동도 학습할 수 있는가



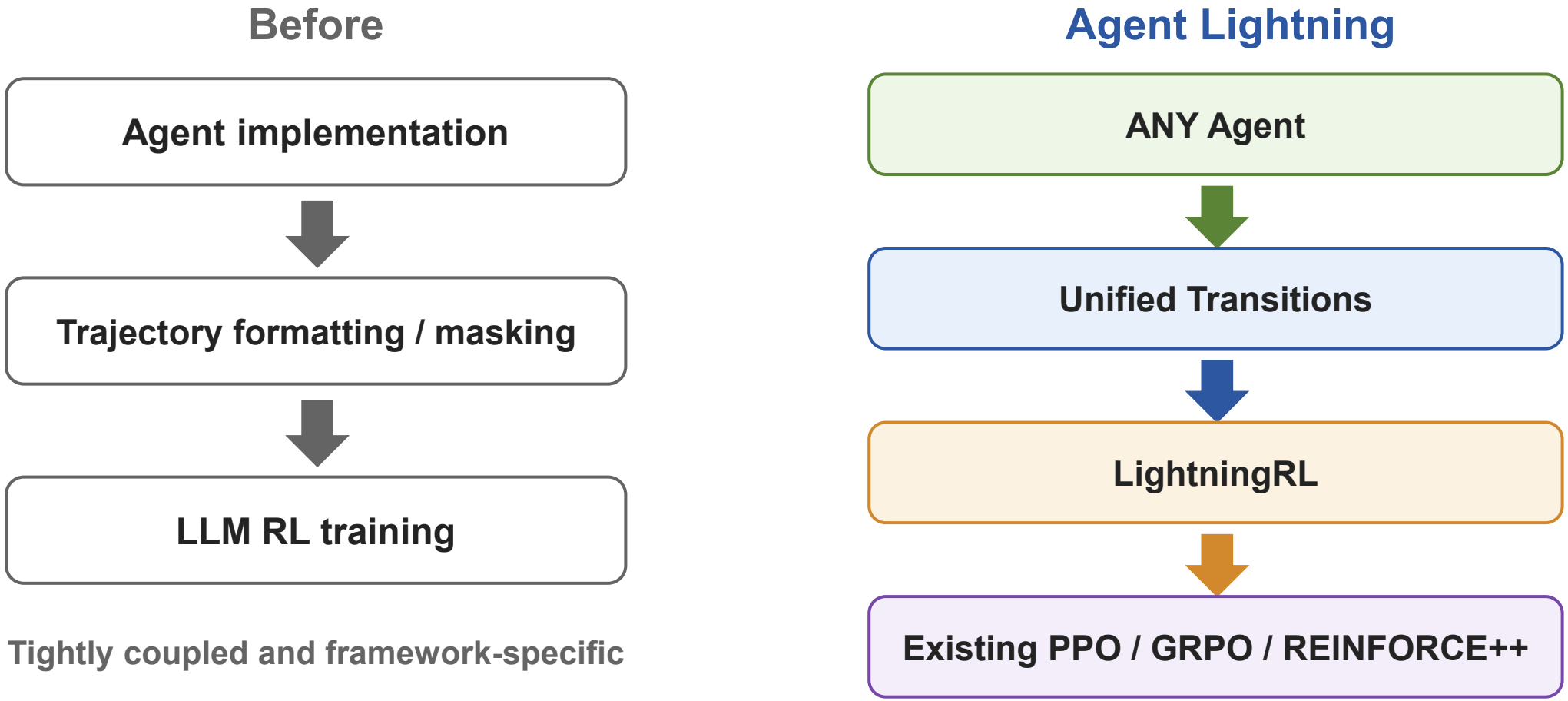
(a) Train reward



(b) Test reward

Can Agent Lightning optimize a tool-using agent, not just a single-call response?

So What Does Agent Lightning Actually Contribute?



Main contribution: a general abstraction + system framework for applying existing LLM RL to arbitrary agents — not a new RL optimizer.

Three Main Contributions

① Unified Data Interface

Abstract agent execution into unified calls/transitions

② LightningRL

Connect multi-call agent episodes to existing single-turn LLM RL

③ Training-Agent Disaggregation

Separate the agent application from GPU/RL training infrastructure

The main contribution is a general framework for training ANY agent with RL, rather than a new RL optimizer

감사합니다.